



**An Experimental Study of Kolmogorov–Arnold
Networks with Convolutional Neural Networks
PROJECT REPORT**

Submitted by

<i>Abhiraj Sardar</i>	<i>12624010001</i>
<i>Anurup Bhowmick</i>	<i>12624010004</i>
<i>Devang Kamdar</i>	<i>12624010014</i>

4th Semester Major Project

of

MASTER OF COMPUTER APPLICATION

DEPARTMENT OF COMPUTER APPLICATIONS

HERITAGE INSTITUTE OF TECHNOLOGY, KOLKATA

MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY,

WEST BENGAL

May, 2026

HERITAGE INSTITUTE OF TECHNOLOGY, KOLKATA

DEPARTMENT OF COMPUTER APPLICATIONS



BONAFIDE CERTIFICATE

Certified that this project report “**An Experimental Study of Kolmogorov–Arnold Networks with Convolutional Neural Networks**” is the bonafide work of our team, students of the MCA 4th semester of **Heritage Institute of Technology, Kolkata**, who carried out the project work under the supervision of **Prof. Sumon Ghosh**.

Prof. (Dr.) Souvik Basu
Head,
Department of Computer Applications
Heritage Institute of Technology, Kolkata

Prof. Sumon Ghosh (Mentor)
Asst. Prof.
Department of Computer Applications
Heritage Institute of Technology, Kolkata

EXAMINER

DECLARATION BY STUDENTS

This is to affirm that our team has authored the report titled “**An Experimental Study of Kolmogorov–Arnold Networks with Convolutional Neural Networks.**” The report does not contain any plagiarized sections from other sources. All information taken from external sources has been properly credited. We affirm that we will bear complete responsibility if any section of the report is identified as plagiarized.

Abhiraj Sardar - 12624010001

Anurup Bhowmick - 12624010004

Devang Kamdar - 12624010014

ACKNOWLEDGEMENT

We are extremely grateful to our project supervisor, **Prof. Sumon Ghosh**, for his guidance, ongoing support, and essential information related to the project. We also express our gratitude to the **Department of Computer Applications, Heritage Institute of Technology**, for granting us convenient access to the project lab and additional resources.

Abhiraj Sardar - 12624010001

Anurup Bhowmick - 12624010004

Devang Kamdar - 12624010014



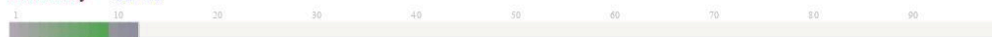
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

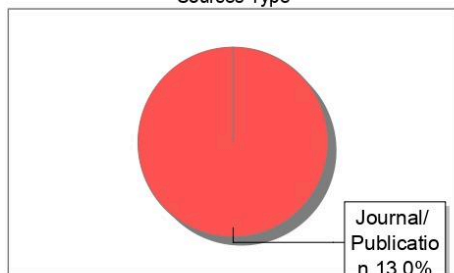
Author Name	Abhiraj Sardar, Anurup Bhowmick, Devang Kamdar
Title	An Experimental Study of Kolmogorov–Arnold Networks with Convolutional Neural Networks
Paper/Submission ID	5734438
Submitted by	mca@heritageit.edu
Submission Date	2026-05-27
Total Pages, Total Words	54, 10610
Document type	Project Report

Result Information

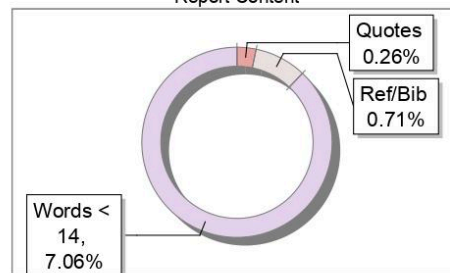
Similarity **13 %**



Sources Type



Report Content



Exclude Information

Quotes	Excluded	Language	English
References/Bibliography	Not Excluded	Student Papers	Yes
Source: Excluded < 14 Words	Not Excluded	Journals & publishers	Yes
Excluded Source	0 %	Internet or Web	No
Excluded Phrases	Not Excluded	Institution Repository	No

Database Selection

A Unique QR Code use to View/Download/Share Pdf File



TABLE OF CONTENTS

Serial No.	Topics	Page No.
1	Abstract	6
2	Introduction	7 - 8
3	Objective	9
4	Features	10
5	Research Motivation	11 - 12
6	Architecture	13 - 22
7	Mathematical Foundation of KAN	23 - 24
8	Limitations of Traditional Fully Connected Layers	25 - 26
9	Dataset Description and Analysis	27 - 30
10	Image Preprocessing Pipeline	31 - 34
11	System Requirements	35 - 36
12	Installation and setup	37 - 38
13	Data Flow and Schema Diagram	39
14	Project Snapshots	40 - 45
15	Selected Codes	46 - 49
16	Future Scope	50 - 51
17	Conclusion	52 - 53
18	References	54

Abstract

This project focuses on comparing a traditional Convolutional Neural Network (CNN) with a modified Convolutional Kolmogorov-Arnold Network (CON-KAN) for Cat and Dog image classification. The main objective of the work is to study the impact of replacing the conventional Fully Connected (FC) layers in CNNs with Kolmogorov-Arnold Network (KAN) layers and to understand how this modification affects the learning capability and overall performance of the model.

The proposed CON-KAN architecture combines the strong feature extraction ability of CNNs with the adaptive learning capability of KANs. In the developed model, convolutional layers are used to extract important visual features from images, while the KAN layer replaces the traditional dense classification layer to learn complex nonlinear relationships more effectively. The architecture includes multiple convolutional layers, pooling operations, adaptive average pooling, flattening operations, linear transformation layers, and a final KAN layer configured with a width of [128, 257, 1], knot value 8, and spline degree 3.

The system was implemented using the PyTorch deep learning framework and trained on the Kaggle “Cat and Dog” dataset containing more than 10,000 labeled images. Various preprocessing techniques such as image resizing, normalization, and tensor conversion were applied to ensure stable and consistent model training. Both the CNN and CON-KAN models were trained under similar experimental settings using the Adam optimizer and Binary Cross Entropy with Logits Loss (BCEWithLogitsLoss) function.

The performance of both models was evaluated using parameters such as training accuracy, validation accuracy, loss analysis, and parameter efficiency. The experimental results show that the CON-KAN model performs competitively while introducing a more flexible and adaptive learning mechanism compared to traditional fully connected layers used in CNNs. The project also demonstrates the practical deployment of the trained model using a FastAPI backend hosted on CPU-based cloud infrastructure.

Overall, this study highlights the potential of integrating Kolmogorov-Arnold Networks into modern deep learning architectures and suggests that hybrid CON-KAN models can serve as an effective alternative for image classification and future computer vision applications.

Introduction

Deep learning has brought major advancements in the field of computer vision by enabling machines to automatically learn and understand complex visual patterns from digital images. With the increase in computational power and the availability of large-scale datasets, deep learning models have achieved impressive results in various image-related tasks such as object detection, image segmentation, facial recognition, medical image analysis, and image classification. Among the different deep learning architectures, convolutional neural networks (CNNs) have become one of the most widely used and successful models because of their strong feature extraction capability and high accuracy in image processing tasks.

Convolutional neural networks are specially designed to process image data by learning important spatial and hierarchical features directly from images. A typical CNN architecture consists of convolutional layers, activation functions, pooling layers, and fully connected layers that work together to identify meaningful patterns from input images. The convolutional layers help in extracting both low-level and high-level features such as edges, textures, contours, and object shapes, while the fully connected layers are mainly responsible for the final classification process. Traditional CNN architectures primarily depend on linear transformations combined with fixed activation functions to learn complex relationships within the data.

Even though CNNs have shown excellent performance in computer vision applications, researchers are continuously exploring new mathematical approaches to improve the flexibility and learning capability of neural networks. One such emerging approach is the Kolmogorov-Arnold Network (KAN), which is considered a promising alternative to traditional Multi-Layer Perceptrons (MLPs). The KAN architecture is inspired by the Kolmogorov-Arnold representation theorem and introduces learnable spline-based functions instead of fixed activation functions. This allows the network to model nonlinear relationships in a more adaptive and flexible manner.

Unlike traditional fully connected neural networks, KANs improve function approximation by learning adaptive mathematical transformations directly from the data. The spline-based learning mechanism enables the network to capture complex nonlinear patterns more efficiently while maintaining better parameter utilization. Due to these advantages, KANs are increasingly being explored in different deep learning applications to improve model performance and learning behavior.

This project focuses on the development and comparative analysis of a hybrid deep learning architecture called a Convolutional Kolmogorov-Arnold Network (CON-KAN) for image classification. In the proposed model, the traditional Fully Connected (FC) layer of a CNN is replaced with a KAN layer to study its effect on the overall learning capability and classification performance of the network. The architecture combines the powerful feature extraction capability of convolutional layers with the adaptive nonlinear learning capability of KAN layers.

The experiments for this project were performed using the Kaggle “Cat and Dog” image dataset, which contains thousands of labeled images belonging to two categories: cats and dogs. Both the traditional CNN model and the proposed CON-KAN model were trained under similar experimental conditions using the PyTorch deep learning framework. The models were evaluated using different performance metrics such as training accuracy, validation accuracy, loss analysis, and parameter efficiency.

The main objective of this study is to analyze whether replacing traditional fully connected layers with KAN layers can improve the flexibility and learning capability of deep learning models while maintaining competitive classification performance. The comparative analysis carried out in this project helps in understanding the practical benefits and limitations of integrating KAN layers into CNN-based architectures.

In addition to model development and experimentation, the project also includes deployment of the trained model using a FastAPI backend and an Angular-based frontend interface. This deployment architecture demonstrates the practical feasibility of running deep learning inference systems in resource-constrained environments using CPU-based cloud deployment platforms.

The experimental results indicate that the proposed CON-KAN architecture performs competitively while introducing a more flexible and mathematically expressive learning mechanism compared to traditional CNN architectures. This study highlights the potential of combining conventional convolutional feature extraction techniques with spline-based KAN learning methods to develop more efficient and adaptive deep learning models for future image classification applications.

Objective

The primary objective of this project is to develop an efficient deep learning-based image classification system capable of accurately distinguishing between cat and dog images. The project focuses on implementing modern neural network architectures for computer vision tasks and utilizes the strong feature extraction capability of Convolutional Neural Networks (CNNs) to learn meaningful visual representations from image data for binary image classification.

One of the key objectives of this work is to understand the architecture and working mechanism of convolutional neural networks, which are widely used in image processing and computer vision applications. The project studies how convolutional layers, pooling operations, activation functions, and classification layers work together to extract hierarchical image features and perform accurate classification tasks.

Another important objective is to explore the concept and functionality of Kolmogorov-Arnold Networks (KANs) as an alternative to traditional fully connected neural network layers. KANs introduce adaptive spline-based nonlinear functions that provide improved flexibility in learning complex relationships from data. This project aims to analyze how integrating KAN layers can improve the learning capability and representational power of deep learning models compared to conventional fully connected architectures.

The proposed system specifically replaces the traditional Fully Connected (FC) layer used in CNNs with a KAN layer to create a hybrid Convolutional Kolmogorov-Arnold Network (CON-KAN). This modification is intended to evaluate whether adaptive spline-based learning can improve classification performance, parameter efficiency, and overall model flexibility.

The project also focuses on performing a comparative analysis between the traditional CNN model and the proposed CON-KAN architecture using the Kaggle Cat and Dog image dataset. Both models are trained and tested under similar experimental conditions to ensure fair performance evaluation. The comparison is carried out using parameters such as training accuracy, validation accuracy, loss analysis, and overall learning behavior.

In addition to model development, the project also aims to build a complete end-to-end deep learning application with frontend, backend, and deployment support. The trained PyTorch model is integrated with a FastAPI backend for prediction services and connected to an Angular-based frontend interface for user interaction. The deployment of the system on CPU-based cloud infrastructure further demonstrates the practical implementation of lightweight deep learning applications in real-world environments.

The overall goal of this study is to investigate the effectiveness of integrating KAN layers into modern deep learning architectures and to explore their potential as an alternative approach for image classification tasks. Through this work, the project contributes toward understanding hybrid CNN-KAN architectures and their future applications in advanced computer vision systems.

Features

1. Deep Learning-Based Image Classification

The proposed system is designed to classify images into Cat and Dog categories using deep learning techniques. The model learns important visual patterns directly from image data, enabling accurate and efficient image classification.

2. Hybrid CON-KAN Architecture

The project introduces a hybrid Convolutional Kolmogorov-Arnold Network (CON-KAN) architecture that combines the feature extraction capability of Convolutional Neural Networks (CNNs) with the adaptive learning capability of Kolmogorov-Arnold Networks (KANs). This integration improves the flexibility and nonlinear representation power of the model.

3. Replacement of Traditional Fully Connected Layers

Instead of using conventional Fully Connected (FC) layers, the proposed architecture utilizes KAN layers for classification. The spline-based learning mechanism of KAN enables the model to learn complex nonlinear relationships more adaptively and efficiently.

4. Automatic Feature Extraction

The convolutional layers automatically extract important image features such as edges, textures, contours, and shapes from the input images. This eliminates the need for manual feature engineering and improves the learning process.

5. Comparative Performance Analysis

The project includes a comparative study between traditional CNN and CON-KAN architectures. Both models are evaluated under similar training conditions using performance metrics such as accuracy, loss analysis, and parameter efficiency.

6. Real-Time Prediction System

The system supports real-time image classification through a FastAPI backend integrated with an Angular frontend interface. Users can upload images and instantly receive prediction results from the trained deep learning model.

7. Lightweight Cloud Deployment

The application is deployed using a lightweight cloud-based architecture where the frontend is hosted on Netlify and the backend with the trained PyTorch model is deployed on Render.com. The deployment is optimized for efficient CPU-based inference in resource-constrained environments.

Research Motivation

The rapid growth of Deep Learning has greatly influenced the field of computer vision, especially in image classification tasks. Among various deep learning models, Convolutional Neural Networks (CNNs) have shown excellent performance in extracting spatial and hierarchical image features. However, even though CNNs are highly effective in feature extraction, most traditional architectures still depend on Fully Connected (Dense) layers or Multi-Layer Perceptrons (MLPs) for final classification. These dense layers often increase the number of trainable parameters, computational complexity, and overall model overhead.

Recent research in neural network architectures has introduced alternative approaches aimed at improving learning efficiency and functional representation. One such emerging architecture is the Kolmogorov-Arnold Network (KAN), which is inspired by the Kolmogorov-Arnold Representation Theorem. Unlike traditional neural networks that mainly rely on fixed activation functions and weighted sums, KAN-based architectures utilize learnable spline-based functional mappings. This allows the model to learn nonlinear relationships in a more adaptive and expressive manner.

The motivation behind this project was to explore whether KAN layers could serve as an effective replacement for traditional fully connected layers in CNN-based image classification systems. Instead of replacing the complete convolutional pipeline, the proposed CON-KAN architecture combines the strengths of both CNN and KAN approaches. The convolutional layers are responsible for extracting important image features, while the final classification stage is replaced with a KAN layer to study its learning capability and classification performance.

A fully KAN-based convolutional architecture was not implemented due to practical computational and deployment limitations. KAN operations are computationally more expensive compared to standard convolution operations, especially in resource-constrained environments. Since the final system was intended for deployment using FastAPI and Render.com on CPU-based infrastructure, the architecture was designed carefully to maintain a balance between experimental research and practical deployment feasibility. Therefore, KAN was integrated only in the final classification stage rather than throughout the entire network.

Another important motivation for this project was the increasing interest in exploring non-traditional neural network architectures beyond conventional CNN-MLP combinations. While much of modern deep learning research focuses on larger models and transformer-based systems, fewer practical studies experimentally evaluate alternative mathematical architectures such as KAN in real-world image classification workflows. This project attempts to contribute toward that research direction by analyzing the behavior of a hybrid CNN-KAN architecture under realistic training and deployment conditions.

The cat vs. dog image classification problem was selected as the experimental benchmark because it is a well-known and balanced dataset suitable for evaluating image classification

architectures. The dataset contains enough visual variation to effectively test feature extraction and classification performance while still remaining computationally manageable for training and deployment.

Overall, the proposed CON-KAN architecture was developed as a research-oriented experimental system to investigate whether replacing traditional dense layers with KAN-based functional mappings can provide an efficient and practical alternative for image classification tasks. The project combines mathematical concepts, deep learning experimentation, and deployment engineering into a unified workflow for studying modern hybrid neural network architectures.

Architecture

Model Architecture

Kolmogorov-Arnold Network (KAN)

Kolmogorov-Arnold Networks (KANs) are a recently introduced neural network architecture inspired by the Kolmogorov-Arnold Representation Theorem. In recent years, KANs have gained attention in the deep learning community because of their ability to model complex nonlinear relationships more flexibly compared to traditional neural network architectures. KANs are considered a promising alternative to conventional Multi-Layer Perceptrons (MLPs) since they replace fixed activation-based learning with adaptive learnable functional mappings.

In traditional neural networks such as Multi-Layer Perceptrons, neurons are connected through weighted matrices followed by predefined activation functions like ReLU, Sigmoid, or Tanh. During training, only the weights and biases are updated while the activation functions remain fixed. Although this method has achieved excellent performance across many machine learning tasks, the fixed nature of activation functions can sometimes limit the network's ability to represent highly complex nonlinear patterns effectively.

Kolmogorov-Arnold Networks introduce a different learning mechanism by placing learnable nonlinear functions on the connections between neurons instead of applying fixed activation functions at the nodes. These learnable functions are generally implemented using spline-based mathematical representations that can dynamically adapt their shapes during training according to the data patterns being learned. This adaptive behavior allows KANs to learn more expressive and flexible feature representations compared to traditional architectures.

The spline-based functional mappings used in KANs improve the network's ability to approximate complex nonlinear functions. Since the activation behavior itself becomes trainable, the model gains additional flexibility and representational power while learning detailed patterns from the input data. As a result, KANs are capable of capturing intricate relationships more efficiently and often with fewer parameters than conventional neural network models.

One of the major advantages of KANs is their parameter efficiency and improved generalization capability. By learning adaptive nonlinear transformations directly from data, KANs can represent complex functions in a more compact and efficient manner. This makes them suitable for applications involving high-dimensional and nonlinear data such as image classification, scientific computing, signal processing, and other advanced machine learning tasks.

Due to their promising performance and mathematical flexibility, KANs are currently being explored in different deep learning domains as an alternative to traditional fully connected neural network layers. In this project, KAN layers are integrated into a Convolutional Neural Network architecture to form a hybrid Convolutional Kolmogorov-Arnold Network

(CON-KAN) for image classification. The purpose of this integration is to study how adaptive spline-based learning can improve the learning capability and classification performance of deep learning models.

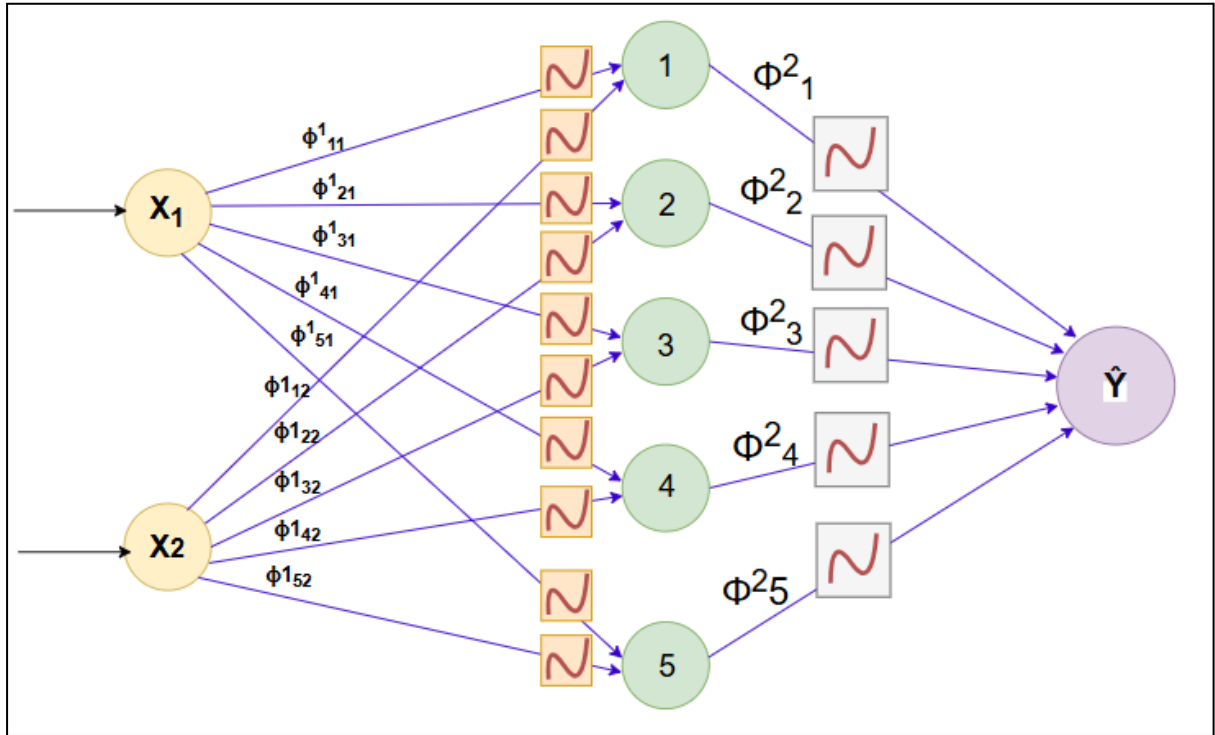


Fig. 1: Multi-Layered Kolmogorov-Arnold Network Architecture

Proposed CON-KAN Architecture

The proposed CON-KAN architecture is a hybrid deep learning model developed by combining the spatial feature extraction capability of convolutional neural networks (CNNs) with the adaptive functional learning behavior of Kolmogorov-Arnold networks (KANs). The architecture was designed as an experimental alternative to conventional CNN-based image classification systems that mainly depend on fully connected dense layers for final prediction. Instead of using a traditional Multi-Layer Perceptron (MLP) classifier, the proposed model integrates a custom KAN-based classification module to study whether adaptive functional mappings can effectively handle image classification tasks.

The overall architecture follows a layered deep learning pipeline consisting of:

- convolutional feature extraction,
- feature normalization,
- dimensional transformation,
- and KAN-based classification.

The system was specifically designed to maintain a balance between research experimentation and practical deployment feasibility in a resource-constrained environment.

Classification Workflow

During inference, the final KAN layer produces a single scalar output representing the binary classification prediction. This output is passed through a Sigmoid activation function to generate probability scores between 0 and 1. A threshold value of 0.5 is then used for the final class prediction:

- $\text{probability} \geq 0.5 \rightarrow \text{Dog}$
- $\text{probability} < 0.5 \rightarrow \text{Cat}$

This design allows the system to work efficiently as a binary classifier while remaining compatible with the BCEWithLogitsLoss function used during training.

Design Motivation and Architectural Trade-Offs

A completely KAN-based convolutional architecture was not implemented in this project because of computational and deployment limitations. KAN operations are computationally more expensive than standard convolution operations, especially when processing high-dimensional image feature maps. Since the final system was intended for deployment in a CPU-based cloud inference environment, replacing the entire convolutional pipeline with KAN operations would have introduced significant training and inference overhead.

The proposed CON-KAN architecture therefore adopts a hybrid strategy:

- CNNs handle efficient spatial feature extraction,
- while KAN performs adaptive functional classification.

This design helps balance:

- experimental neural architecture research,
- computational feasibility,
- deployment practicality,
- and classification performance.

Although the proposed architecture does not currently outperform highly optimized conventional CNN classifiers, experimental observations showed that the model achieved comparatively competitive classification performance while introducing a fundamentally different functional learning mechanism. These results suggest that KAN-based classification remains a promising research direction for future exploration in deep learning and computer vision systems.

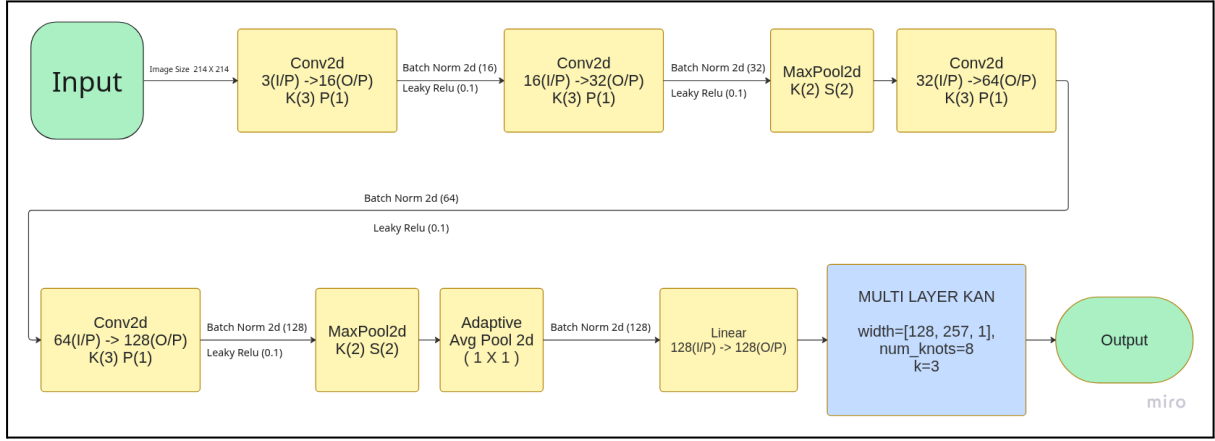


Fig. 2: Convolutional Neural network Architecture

Convolutional Feature Extraction Pipeline

The first stage of the proposed architecture consists of a CNN-based feature extraction module responsible for learning hierarchical spatial representations from input images. The model accepts RGB images resized to 224 x 224 dimensions and processes them through multiple convolutional layers designed to capture meaningful visual patterns from the dataset.

The convolutional pipeline includes:

- Convolutional Layers
- Batch Normalization
- LeakyReLU Activation Functions
- Max Pooling Operations
- Adaptive Average Pooling

The feature extraction sequence is structured as follows:

- Conv2D (3 - > 16)
- BatchNorm2D
- LeakyReLU
- Conv2D (16 - > 32)
- BatchNorm2D
- LeakyReLU
- MaxPool2D
- Conv2D (32 - > 64)
- BatchNorm2D
- LeakyReLU
- Conv2D (64 - > 128)
- BatchNorm2D
- LeakyReLU
- MaxPool2D
- AdaptiveAvgPool2D

The convolutional layers are responsible for learning both low-level and high-level image features such as edges, textures, contours, shapes, and spatial patterns. Batch Normalization

layers are added after convolution operations to improve training stability and help the model converge more efficiently. LeakyReLU activation functions are used to reduce the dead neuron problem that can occur with standard ReLU activations.

Max Pooling layers gradually reduce the spatial dimensions of the feature maps, lowering computational complexity while retaining important feature information. Finally, the Adaptive Average Pooling layer converts the extracted feature maps into a compact representation suitable for the classification stage.

The model training process uses the Adam optimizer with a learning rate of 0.0003 and a weight decay value of 1×10^{-4} . Adam was selected because of its adaptive learning capability and stable optimization behavior during deep neural network training. Weight decay regularization was also incorporated to reduce overfitting and improve the generalization capability of the model.

Feature Flattening and Pre-KAN Transformation

After the feature extraction stage, the pooled feature maps are flattened into a one-dimensional vector representation. Before passing these extracted features into the KAN classifier, an intermediate preprocessing stage called the Pre-KAN Transformation Layer is applied.

The Pre-KAN module consists of:

- BatchNorm1D
- Linear Transformation
- Tanh Activation Function

This stage performs feature normalization and stabilizes the feature distribution before the data enters the KAN module. The intermediate linear transformation reshapes the extracted CNN feature representation into a form that is more suitable for spline-based functional approximation inside the KAN architecture.

The Tanh activation function was intentionally selected because the KAN implementation performs better when input values remain within a bounded range. Since spline-based functional approximation is sensitive to input distribution, this normalization stage helps improve training stability, functional approximation behavior, and overall learning performance.

KAN-Based Classification Layer

The final classification stage of the architecture is implemented using a custom-built MultiKAN module developed specifically for this project. Unlike conventional dense layers that mainly rely on weighted matrix multiplication, the KAN classifier performs learning using spline-based functional mappings inspired by the Kolmogorov-Arnold representation theorem.

The implemented MultiKAN module utilizes:

- Custom B-spline Basis Computation
- Learnable Spline Coefficients
- Knot-Based Interval Partitioning
- Functional Aggregation Operations

The KAN architecture used in this project follows the width configuration:

[128, 257, 1]

where:

- 128 represents the input feature dimension obtained from the CNN feature extractor.
- 257 represents the intermediate functional representation space.
- 1 represents the final binary classification output.

The architecture width was selected experimentally after testing multiple configurations during training. The selected structure showed comparatively better classification performance and more stable convergence behavior compared to smaller configurations.

The KAN implementation further utilizes:

- Spline Degree = 3 (Cubic Splines)
- Number of Knots = 8

These hyperparameters were also selected experimentally based on observed training stability and functional approximation performance. Cubic splines provide smooth functional transitions while maintaining manageable computational complexity.

Within each KAN layer, B-spline basis functions are computed over the input feature dimensions, and learnable spline coefficients are optimized during training using backpropagation. Instead of learning only scalar weights between neurons, the network learns adaptive mathematical transformations capable of approximating complex nonlinear relationships between extracted image features and output classes.

The final output layer produces a binary classification prediction indicating whether the input image belongs to the cat or dog category.

System Architecture

The proposed system follows a client-server architecture designed for real-time cat and dog image classification using the CON-KAN deep learning model. The system mainly consists of three components: the user device, the frontend server, and the backend server integrated with the trained CON-KAN model. The frontend application, developed using Angular and Vanilla CSS, provides a simple and interactive interface that allows users to upload images through a web browser. The backend is implemented using FastAPI, which manages API requests, image preprocessing, and model inference operations.

When a user uploads an image through the frontend interface, the image is sent to the backend server using an HTTP POST request through the prediction API endpoint. The

backend first preprocesses the uploaded image and then passes it to the trained CON-KAN model for classification. Based on the learned image features, the model predicts whether the uploaded image belongs to the cat or dog category. The prediction result is then returned to the user through an HTTP response and displayed on the frontend interface in real time. The frontend application is deployed on Netlify, while the FastAPI backend, along with the trained PyTorch model, is deployed on Render.com, enabling lightweight CPU-based inference and scalable real-time prediction.

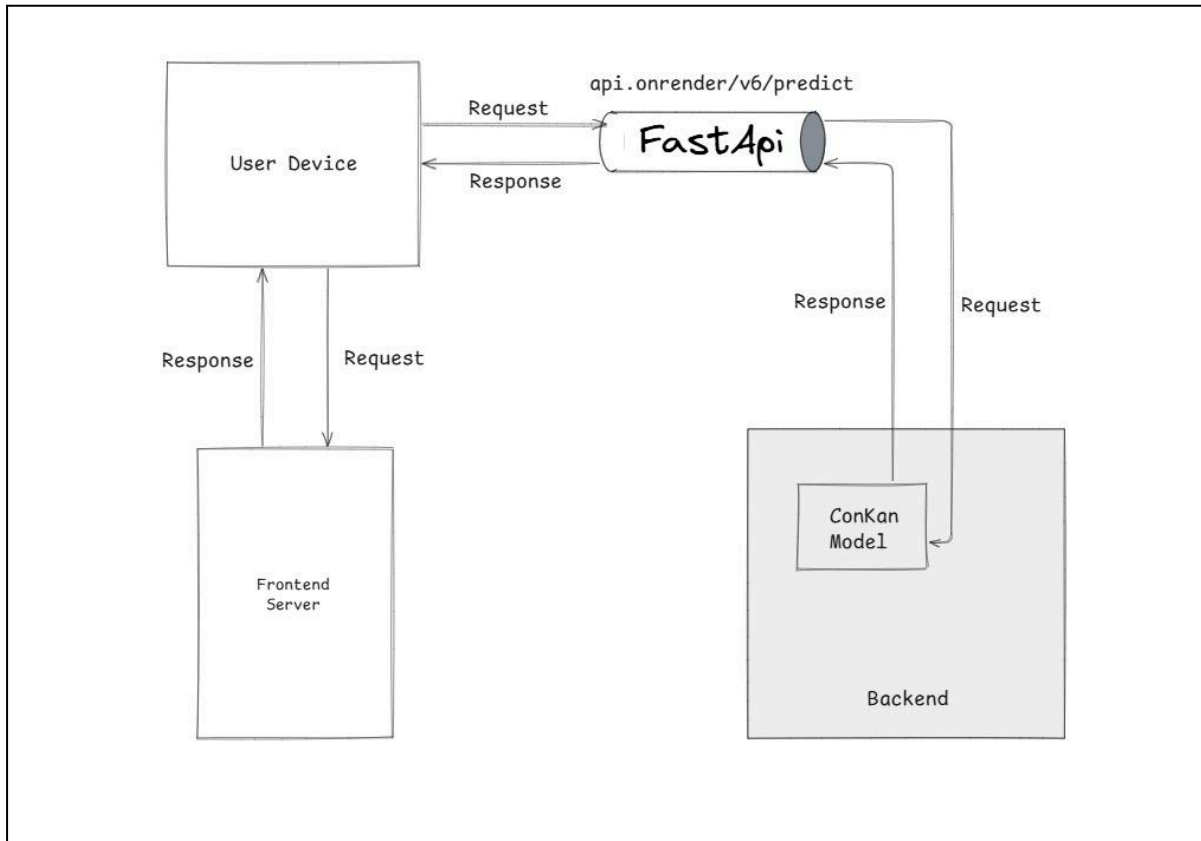


Fig. 3: System Architecture

Why was FastAPI Selected ?

FastAPI was selected as the backend framework for the CON-KAN project because of its high performance, lightweight design, and smooth compatibility with Python-based deep learning workflows. Since the proposed system performs real-time image classification using a PyTorch model, it was important to choose a backend framework capable of handling inference requests efficiently while maintaining low overhead and fast response times.

One of the key advantages of FastAPI is its support for asynchronous request handling, which improves scalability and responsiveness when managing multiple API requests simultaneously. The framework also offers automatic request validation and structured response handling through Pydantic models, making the API more reliable, maintainable, and easier to manage during development.

Another important reason for selecting FastAPI was its built-in support for automatic API documentation generation using OpenAPI and Swagger UI. This feature simplified API testing, endpoint validation, and backend debugging throughout the development process.

Compared to Django, FastAPI is much more lightweight and better suited for machine learning inference systems that do not require a large monolithic web framework structure. Similarly, when compared to Flask, FastAPI provides stronger type validation, better support for asynchronous operations, and a cleaner API architecture for production-oriented AI applications.

Since the complete machine learning pipeline was developed using Python and PyTorch, FastAPI provided an efficient and practical solution for integrating model inference into a REST API-based deployment workflow.

Why was Angular selected?

Angular was selected as the frontend framework for the CON-KAN project because of its structured architecture, strong TypeScript support, and modular component-based development approach. The project required a scalable frontend system capable of managing image upload workflows, prediction visualization, informational pages related to the KAN architecture, and interactive UI components within a single framework.

One of the major advantages of Angular is its well-organized project structure, which simplifies the development and management of multiple frontend modules while improving long-term maintainability. The framework also provides built-in features such as routing, component communication, dependency injection, and seamless API integration, making it suitable for building structured AI-based web applications.

The strong integration of TypeScript in Angular further improves code readability, scalability, and type safety. This helps maintain consistency across frontend components and ensures smoother communication between the frontend and backend API layers.

The frontend system developed for this project includes:

- image upload interfaces,
- prediction visualization,
- confidence score display,
- informational pages related to KAN,
- and UI animation components.

Angular's modular architecture made it easier to organize these functionalities within a clean and scalable frontend workflow.

Instead of selecting a lightweight frontend library focused mainly on UI rendering, Angular was chosen because it provides a more complete frontend ecosystem suitable for structured application development, scalability, and long-term project maintenance.

Why was Render.com selected?

Render.com was selected as the backend deployment platform because it provides a simple and efficient cloud hosting environment for Python-based API services and machine learning inference systems. Since the backend of the project was developed using FastAPI and

PyTorch, it was important to use a platform that could support lightweight API hosting with minimal infrastructure setup and configuration.

One of the key advantages of Render.com is its smooth deployment workflow through GitHub integration. This simplified the process of backend deployment, dependency installation, environment configuration, and application updates throughout the development cycle.

Render.com also provides several useful features such as:

- automatic HTTPS support,
- environment variable management,
- continuous deployment integration,
- and simplified server configuration.

These features made the platform suitable for deploying the FastAPI inference server without requiring complex DevOps or cloud infrastructure management.

Another important reason for choosing Render.com was its support for lightweight CPU-based deployment environments. Since the project was developed under resource constraints, the backend inference system needed to operate efficiently without relying on expensive GPU cloud infrastructure.

Although KAN-based architectures introduce additional computational overhead compared to traditional CNN classifiers, the deployment results demonstrated that the proposed CON-KAN system could still function effectively within a practical CPU-based cloud environment.

Why was Netlify Selected ?

Netlify was selected for frontend deployment because it provides fast and efficient hosting for static web applications along with a simplified deployment workflow and seamless GitHub integration. Since the Angular frontend operates independently from the FastAPI backend inference server, Netlify offered a practical solution for hosting the frontend separately while still allowing smooth communication with backend APIs.

One of the major advantages of Netlify is its continuous deployment pipeline, which automatically rebuilds and deploys frontend updates whenever changes are pushed to the connected GitHub repository. This greatly simplified frontend deployment and reduced manual configuration during development.

Netlify also provides several useful features, such as:

- global CDN optimization,
- automatic HTTPS configuration,
- fast static asset delivery,
- and reliable frontend hosting performance.

These features helped improve frontend responsiveness and overall user experience while reducing deployment complexity.

Since the frontend mainly handles:

- image uploads,
- prediction visualization,
- informational content,
- and UI rendering,

Netlify's lightweight static hosting model was highly suitable for the project requirements.

The combination of Netlify for frontend deployment and Render.com for backend inference hosting created a decoupled deployment architecture where the frontend and backend systems could operate independently while maintaining efficient API-based communication

Why was PyTorch selected instead of TensorFlow?

PyTorch was selected for the CON-KAN project because of its flexibility, dynamic computation graph, and strong support for research-oriented deep learning development. Since the project involved implementing a custom KAN-based architecture with spline-based functional mappings, PyTorch provided a more intuitive and flexible environment for developing and experimenting with custom neural network layers.

Another major advantage of PyTorch is its simpler debugging workflow, modular design, and seamless integration with Python, which made experimentation and model tuning easier during development. Compared to TensorFlow, PyTorch is generally more suitable for rapid prototyping and custom architecture research.

Additionally, PyTorch integrated efficiently with the FastAPI backend used for deployment, simplifying the inference pipeline and model serving workflow for the proposed CON-KAN system.

Mathematical Foundation of KAN

The theoretical foundation of the proposed CON-KAN architecture is based on the Kolmogorov-Arnold Representation Theorem, a mathematical concept that explains how complex multivariable functions can be represented using combinations of simpler one-dimensional functions. This theorem serves as the core inspiration behind Kolmogorov-Arnold Networks (KANs), which aim to introduce an alternative learning mechanism compared to traditional artificial neural networks (ANNs).

Traditional neural networks, especially multi-layer perceptrons (MLPs), learn through weighted linear transformations followed by fixed activation functions such as ReLU, sigmoid, or tanh. In these architectures, learning mainly occurs by updating connection weights between neurons during backpropagation. Although this approach has achieved excellent performance across modern deep learning applications, it often leads to highly parameterized dense architectures that can become computationally expensive and difficult to interpret.

The Kolmogorov-Arnold Representation Theorem suggests that any continuous multivariable function can theoretically be expressed as a finite composition of continuous one-dimensional functions combined with addition operations. In the context of neural network research, this theorem introduces the idea that complex mappings can potentially be approximated using adaptive functional transformations rather than relying entirely on conventional neuron-weight interactions.

KAN architectures apply this concept by replacing traditional fixed neuron activations with learnable functional mappings. Instead of applying a predefined activation function after a weighted sum, KAN layers learn spline-based transformations that adapt dynamically during training. This enables the network to model nonlinear relationships through trainable mathematical functions rather than depending only on dense parameter interactions.

In the proposed CON-KAN model, the KAN layer is implemented using B-spline basis functions. B-splines are piecewise polynomial functions commonly used in approximation theory and curve fitting because of their flexibility, smoothness, and numerical stability. During training, spline coefficients are learned automatically, allowing the model to approximate nonlinear relationships between extracted image features and classification outputs more effectively.

The implemented KAN layer consists of:

- learnable spline coefficients,
- knot-based spline intervals,
- basis function generation,
- and functional aggregation operations.

The model computes spline basis values over the input feature dimensions and combines them using trainable coefficients to generate classification outputs. This approach is significantly different from standard fully connected layers, where outputs are primarily generated through matrix multiplication followed by fixed nonlinear activations.

Another important aspect of the KAN architecture is the selection of spline degree and knot count. In this project, the spline degree was experimentally selected as degree 3, corresponding to cubic spline functions. Cubic splines provide a good balance between smooth functional approximation and computational complexity, making them suitable for stable learning behavior. Similarly, the number of knots was experimentally chosen as 8 after evaluating training performance across multiple configurations. The knot count directly influences the flexibility and expressive capability of the learned functional mappings.

The proposed CON-KAN architecture does not completely replace convolutional operations with KAN-based computation. Instead, KAN is integrated only into the final classification stage of the network. The convolutional layers continue to perform spatial feature extraction because CNNs remain highly effective at capturing local image patterns such as edges, textures, contours, and object structures. The extracted feature representations are then passed into the KAN classifier for spline-based functional classification.

This hybrid design was mainly adopted because fully KAN-based convolutional architectures are computationally expensive and difficult to deploy efficiently in resource-constrained environments. By combining CNN-based feature extraction with KAN-based classification, the proposed architecture attempts to balance experimental neural architecture research with practical deployment feasibility.

The mathematical motivation behind KAN research is important because it explores whether adaptive functional transformations can act as an effective alternative to traditional dense neural representations. Although KAN architectures are still relatively new and remain an active area of research, they introduce a different perspective on neural learning by emphasizing learnable functional approximation instead of relying solely on weighted activations.

Within the scope of this project, the mathematical foundation of KAN serves not only as a theoretical concept but also as the primary research motivation behind the proposed CON-KAN architecture. The integration of spline-based functional learning into an image classification pipeline demonstrates how mathematical approximation theories can be practically applied within modern deep learning systems.

Limitations of Traditional Fully Connected Layers

Fully Connected (Dense) layers have traditionally been one of the core components of Artificial Neural Networks (ANNs) and modern Convolutional Neural Networks (CNNs). In most image classification systems, convolutional layers are responsible for extracting hierarchical visual features, while dense layers perform the final classification by mapping those extracted features to output classes. Although this architecture has achieved remarkable success in deep learning applications, fully connected layers still have several theoretical and practical limitations that motivate research into alternative architectures such as Kolmogorov-Arnold Networks (KANs).

One of the major limitations of fully connected layers is the rapid increase in trainable parameters. In dense architectures, every neuron in one layer is connected to every neuron in the next layer. As the feature dimensions increase, the number of parameters grows significantly, leading to higher memory consumption, increased computational cost, and slower optimization during training. In many deep learning systems, the final dense layers alone contribute a large portion of the total model parameters despite not participating directly in spatial feature extraction.

Another important limitation is the dependency on weighted linear transformations followed by fixed nonlinear activation functions. Traditional dense neural networks mainly learn through matrix multiplication combined with activation functions such as ReLU, Sigmoid, or Tanh. While this mechanism works effectively in many applications, it limits the network to learning through parameterized weight adjustments instead of directly learning adaptive mathematical functions. This can reduce representational flexibility in complex nonlinear learning scenarios.

Fully connected layers may also contribute to overfitting, especially when models are trained on relatively limited datasets. Since dense layers contain a large number of trainable parameters, the model can become highly specialized to training samples and may struggle to generalize well on unseen data. This becomes particularly relevant in image classification systems where convolutional layers may already extract strong feature representations, while the dense classifier introduces additional parameter complexity.

Another challenge associated with traditional dense architectures is reduced interpretability. The decision-making behavior of fully connected networks is distributed across numerous weighted interactions, making it difficult to clearly understand how specific functional relationships are learned. Although modern deep learning systems achieve strong predictive performance, the internal mathematical behavior of dense neuron interactions often remains difficult to analyze directly.

From a deployment perspective, dense layers can also introduce computational limitations. Large parameter-heavy classifiers increase inference cost, memory usage, and overall computational overhead, which can become problematic in resource-constrained environments such as CPU-based servers, edge devices, or lightweight inference systems. As deep learning applications move increasingly toward real-world deployment, the need for

more efficient alternatives to computationally expensive dense architectures have become an active research area.

These limitations have encouraged researchers to explore alternative neural computation approaches capable of representing complex relationships through more adaptive functional learning mechanisms. Kolmogorov-Arnold networks represent one such emerging direction. Instead of relying entirely on weighted neuron interactions, KAN architectures use learnable spline-based functional transformations to approximate nonlinear relationships between inputs and outputs. This introduces a fundamentally different approach to neural learning compared to conventional dense-layer architectures.

Within the proposed CON-KAN architecture, these limitations served as the primary motivation for experimentally replacing the traditional classification stage with a KAN-based functional mapping layer. Rather than removing convolutional operations completely, the architecture preserves CNN-based spatial feature extraction while integrating KAN into the final classification stage. This hybrid approach attempts to combine the strong visual feature learning capability of CNNs with the adaptive functional learning behavior of KAN.

Experimental observations during implementation also revealed some of the practical trade-offs associated with alternative architectures. Compared to conventional CNN classifiers, the KAN-based system required more careful hyperparameter tuning and introduced higher computational complexity during training. Training stability occasionally became more challenging due to the spline-based functional learning mechanism. However, despite these additional complexities, the CON-KAN model achieved classification performance relatively close to traditional CNN-based systems, indicating that KAN-based classification remains a promising research direction for future exploration.

Although traditional fully connected layers continue to play a major role in modern deep learning systems, their theoretical and computational limitations continue to motivate research into more expressive and efficient neural representations. The integration of KAN within the CON-KAN project therefore represents an experimental effort to study whether functional learning architectures can serve as practical alternatives to conventional dense neural classifiers in image classification workflows.

Dataset Description and Analysis

The performance and generalization capability of any deep learning model depend heavily on the quality, structure, and diversity of the dataset used during training. For the proposed CON-KAN architecture, the Cats vs Dogs dataset obtained from Kaggle was selected as the primary dataset for experimentation and evaluation. The dataset provides a balanced binary image classification problem that is computationally manageable while still containing sufficient visual variation to evaluate the behavior of hybrid CNN-KAN architectures effectively.

The selected dataset was sourced from:

tongpython/cat-and-dog

The dataset contains labeled images belonging to two categories:

- Cats
- Dogs

The dataset was chosen because it represents one of the most widely recognized benchmark problems in introductory and intermediate-level computer vision research. Despite being comparatively simpler than large-scale datasets such as ImageNet, the Cats vs Dogs dataset still contains substantial variation in the following:

- pose,
- lighting conditions,
- background complexity,
- image orientation,
- object scale,
- fur texture,
- and viewpoint diversity.

These variations make the dataset suitable for evaluating both feature extraction capability and classification behavior in deep learning architectures.

Dataset Distribution

The dataset used in this project consists of:

Dataset Split	Cat Images	Dog Images	Total
Training Set	4001	4006	8007
Testing Set	1012	1013	2025

The near-balanced class distribution helps reduce classification bias during training and allows the model to learn features from both categories more uniformly. Balanced datasets are particularly important in binary classification systems because highly imbalanced data may cause the network to become biased toward dominant classes.

Nature of the Dataset

The images within the dataset vary significantly in:

- image quality,
- object positioning,
- resolution,
- background clutter,
- and lighting conditions.

Some images contain close-up animal faces, while others contain full-body images with highly complex backgrounds. This diversity introduces additional challenges for feature extraction and classification, making the dataset appropriate for evaluating the robustness of the proposed CON-KAN architecture.

Another important characteristic of the dataset is the visual similarity between certain cat and dog breeds. Some dog images contain fur patterns and facial structures visually similar to cats, while certain cat images may contain poses or textures that resemble smaller dog breeds. Such similarities increase classification difficulty and encourage the network to learn deeper feature representations rather than relying only on superficial image characteristics.

Image Resizing and Standardization

The original dataset images contain varying spatial resolutions and aspect ratios. Since deep learning models require consistent tensor dimensions during batch processing, all images were resized to:

224×224

This resolution was selected because:

- it provides sufficient visual detail for feature extraction,
- remains computationally manageable during training,
- and aligns with commonly used CNN input dimensions in modern computer vision architectures.

Standardizing image dimensions also improves batch processing efficiency and simplifies convolutional computation throughout the network pipeline.

Data Preprocessing and Normalization

To improve training stability and model convergence, image normalization was applied using standard ImageNet normalization statistics:

Mean = [0.485, 0.456, 0.406]

Std = [0.229, 0.224, 0.225]

Normalization transforms image pixel distributions into a more stable numerical range, helping the model learn more efficiently during optimization. Since the convolutional

backbone learns hierarchical feature representations, normalized input distributions improve gradient behavior and reduce instability during training.

Images were additionally converted into tensor representations before being passed into the model pipeline.

Data Augmentation Strategy

To improve model generalization and reduce overfitting, data augmentation techniques were applied during training. The implemented augmentation strategy included:

- Random Rotation (± 10 degrees)

The purpose of augmentation was to expose the network to slight variations in image orientation and spatial positioning. This encourages the model to learn more generalized feature representations rather than memorizing fixed image patterns from the training set.

Unlike aggressive augmentation pipelines involving heavy transformations, the augmentation strategy in this project was intentionally kept lightweight to preserve image semantics while still improving robustness.

Data Loading and Batch Processing

The dataset was loaded using PyTorch DataLoader utilities with shuffled batch processing enabled during training. Data shuffling ensures that the model does not learn unintended ordering patterns from the dataset and improves stochastic optimization behavior during gradient descent.

The training process utilized:

- batch size = 32

Batch-based training improves GPU utilization efficiency and stabilizes gradient updates during optimization.

Importance of Dataset Selection in CON-KAN

The dataset plays a particularly important role in this project because the objective was not simply to achieve high classification accuracy, but also to evaluate the behavior of a hybrid CNN-KAN architecture under practical image classification conditions. The Cats vs Dogs dataset provided a controlled yet sufficiently challenging environment for studying

- CNN-based feature extraction,
- KAN-based classification behavior,
- training stability,
- convergence characteristics,
- and deployment feasibility.

Because KAN architectures remain relatively new and experimentally oriented, selecting a computationally manageable dataset was important for balancing research experimentation with available training resources and deployment limitations.

Overall, the dataset served as an effective benchmark for evaluating the proposed CON-KAN architecture and provided the necessary visual diversity, class balance, and computational feasibility required for conducting the experimental objectives of the project.

Image Preprocessing Pipeline

Image preprocessing is one of the most important stages in any deep learning-based computer vision system because raw image data cannot be efficiently processed by neural networks without proper transformation and normalization. In the proposed CON-KAN architecture, a structured preprocessing pipeline was implemented to ensure that input images remain consistent, numerically stable, and suitable for both convolutional feature extraction and KAN-based classification.

The preprocessing workflow was designed to perform the following:

- image resizing,
- data augmentation,
- tensor conversion,
- normalization,
- and batch preparation

before passing images into the neural network pipeline.

Need for Image Preprocessing

The original dataset images contain the following variations:

- resolutions,
- aspect ratios,
- lighting conditions,
- color distributions,
- and orientations.

Deep learning models require fixed-dimensional tensor inputs for efficient batch processing and convolution operations. Without preprocessing, inconsistent image dimensions and unnormalized pixel distributions could lead to unstable training behavior and poor convergence performance.

Additionally, since the proposed architecture integrates a spline-based KAN classifier, maintaining stable feature distributions becomes even more important. Functional approximation layers are highly sensitive to input value distributions, making preprocessing a critical part of the overall learning pipeline.

Image Resizing

The first preprocessing step involved resizing all input images to:

224 x 224 This resizing operation standardizes image dimensions across the dataset and ensures compatibility with the convolutional architecture.

The selected resolution provides several advantages:

- sufficient visual detail for feature extraction,
- manageable computational complexity,
- stable GPU memory utilization,
- and compatibility with common CNN design practices.

Smaller image sizes may reduce feature quality, while significantly larger dimensions would unnecessarily increase computational cost and training time for the scope of this project.

Data Augmentation

To improve the model's generalization capability and reduce overfitting, lightweight data augmentation was applied during training.

The implemented augmentation strategy included:

- `RandomRotation(10)`

This operation randomly rotates training images within a small angular range. The purpose of this augmentation is to expose the model to slight orientation variations during training so that it learns more generalized spatial features instead of memorizing fixed image patterns.

The augmentation strategy was intentionally kept minimal because excessive transformations may distort important image structures and negatively affect training stability, especially in hybrid architectures involving spline-based functional learning.

No augmentation was applied during testing or inference to maintain consistent evaluation conditions.

Tensor Conversion

After augmentation and resizing, images were converted into tensor representations using:

- `transforms.ToTensor()`

This step converts image pixel values into PyTorch tensor format and scales pixel intensities from

`[0, 255]`

to:

`[0, 1]`

Tensor conversion enables efficient GPU computation and ensures compatibility with PyTorch-based neural network operations.

Image Normalization

Normalization was applied using standard ImageNet normalization statistics:

- Mean = [0.485, 0.456, 0.406]
- Std = [0.229, 0.224, 0.225]

Normalization is an essential preprocessing step because it stabilizes input distributions and improves optimization during training. By transforming image channels into a normalized numerical range, gradient updates become more stable and model convergence improves significantly.

The selected normalization values are commonly used in computer vision applications and work effectively with standard CNN feature learning behavior. Since the proposed architecture combines convolutional feature extraction with KAN-based classification, maintaining controlled feature distributions was especially important for stable spline-based functional approximation.

Training and Testing Transformation Pipelines

Separate preprocessing pipelines were implemented for:

- training,
- and testing/inference.

The training pipeline included augmentation operations, while the testing pipeline focused only on deterministic preprocessing operations.

Training Transformations

The training preprocessing pipeline performs:

- Image resizing
- Random rotation augmentation
- Tensor conversion
- Normalization

This pipeline improves model generalization and robustness during optimization.

Testing and Inference Transformations

The testing pipeline performs:

- Image resizing
- Tensor conversion
- Normalization

No augmentation is applied during inference because predictions should remain deterministic and reproducible.

Inference Preprocessing Workflow

During deployment, uploaded images pass through a dedicated preprocessing function before model inference.

The inference workflow includes:

- reading uploaded image bytes,
- RGB conversion,
- resizing,
- tensor transformation,
- normalization,
- and batch dimension generation.

The image is first converted into RGB format to ensure consistent channel representation across all inputs. The transformed tensor is then expanded using:

- `unsqueeze(0)`

to create a batch dimension compatible with the neural network input structure.

The final tensor is then transferred to the appropriate computation device before inference.

Importance of Preprocessing in CON-KAN

Preprocessing played an especially important role in the proposed CON-KAN architecture because KAN-based functional mappings are more sensitive to unstable feature distributions compared to standard dense neural layers. Proper normalization and feature stabilization helped improve:

- convergence stability,
- spline approximation behavior,
- gradient consistency,
- and overall classification reliability.

The preprocessing pipeline therefore served not only as a standard computer vision preparation stage but also as an important component for maintaining stable functional learning behavior within the hybrid CNN-KAN system.

Overall, the implemented preprocessing workflow successfully standardized image data, improved training stability, reduced overfitting risk, and supported efficient inference deployment within the proposed image classification framework.

System Requirements

Software Requirements

Component	Technology Used	Description
Frontend Framework	Angular 13	Used for developing the user interface and handling frontend interactions.
Styling	Vanilla CSS	Used for designing and styling the web application interface.
Backend Framework	FastAPI	Used for building REST APIs and handling prediction requests.
ASGI Server	Uvicorn	Used to run the FastAPI backend server efficiently.
Deep Learning Framework	PyTorch	Used for developing, training, and deploying the CON-KAN model.
Image Processing	Pillow (PIL)	Used for image loading and preprocessing operations.
Numerical Computation	NumPy	Used for numerical operations and tensor manipulation.
Data Visualization	Matplotlib	Used for plotting graphs such as accuracy and loss curves.
Machine Learning Utilities	Scikit-learn	Used for evaluation metrics and performance analysis.
API Data Handling	Python Multipart	Used for handling image file uploads through APIs.
Development Language	Python	Main programming language used for backend and model development.
Dataset Source	Kaggle Dataset	Used for training and testing the image classification model.
Frontend Deployment	Netlify	Used for hosting the frontend application.
Backend Deployment	Render.com	Used for deploying the FastAPI backend and trained model.

Hardware Requirements

Component	Specification
Processor	Intel i5 / Ryzen 5 or higher
RAM	Minimum 8 GB
Storage	Minimum 10 GB free space
GPU	NVIDIA GPU (for model training)
Internet Connection	Required for deployment and API communication
Operating System	Windows / Linux / macOS

Installation and setup

Installation Steps for CON-KAN

The following steps describe the installation and setup process for the CON-KAN project. The system is divided into two major components:

- Frontend Application
- Backend API and Model Server

1. Frontend Installation

The frontend of the project is developed using Angular and provides the user interface for image upload and prediction visualization.

Step 1: Clone the Repository

Clone the project repository using the following command:

```
git clone <repository-url>
cd Frontend
cd model-frontend
```

Step 2: Install Project Dependencies

Initialize and install all required frontend dependencies using npm:

```
npm install
```

Step 3: Run the Frontend Application

Start the Angular development server using:

```
ng serve
```

After successful execution, the frontend application will start at:

```
http://localhost:4200
```

2. Backend Installation

The backend of the project is developed using FastAPI and integrates the trained PyTorch CON-KAN model for real-time inference.

Step 1: Clone the Repository

Clone the backend repository using the following commands:

```
git clone <repository-url>
cd KAN-Fast-API
```

Step 2: Create a Virtual Environment

Create a Python virtual environment using:

```
python -m venv . venv
```

Ensure that Python 3.10.0 is installed before creating the environment.

Step 3: Activate the Virtual Environment

Activate the virtual environment using the appropriate command based on the operating system:

For Windows

```
.venv\Scripts\activate
```

For macOS/Linux

```
source .venv/bin/activate
```

Step 4: Install Required Dependencies

Upgrade pip and install all required backend dependencies:

```
pip install --upgrade pip
pip install -r requirements.txt
```

Step 5: Verify the Installation

Verify whether PyTorch and FastAPI have been installed successfully using:

```
python -c "import torch; print(f'PyTorch version: {torch.__version__}')"
python -c "import fastapi; print(f'FastAPI version: {fastapi.__version__}')
```

Step 6: Run the Backend Application

Start the FastAPI application server using:

```
uvicorn app.main:app --reload
```

After successful execution, the backend server will start at:

```
http://localhost:8000
```

Data Flow and Schema Diagram

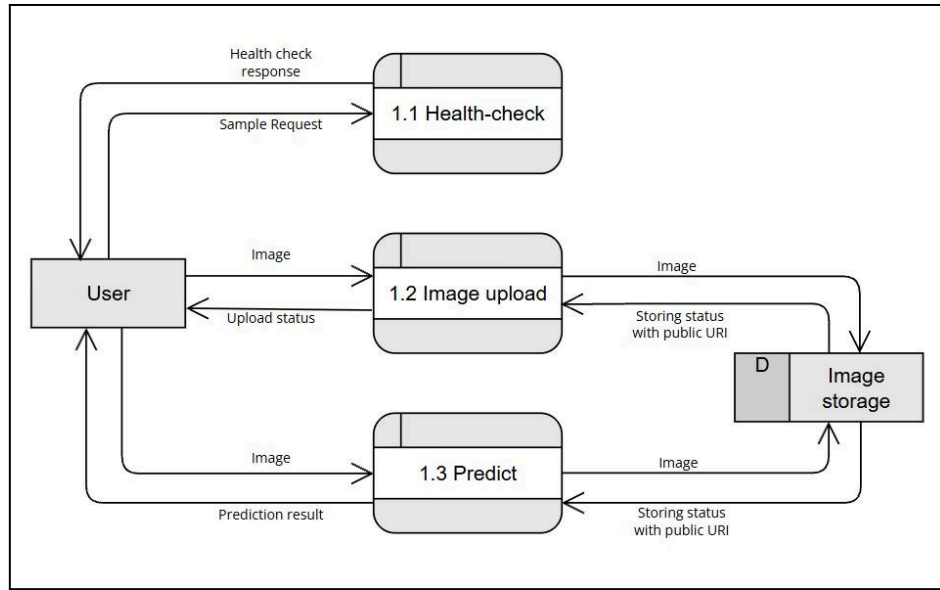


Fig. 4: DFD for Backend Architecture

The above Data Flow Diagram (DFD) illustrates the flow of data within the proposed CON-KAN image classification system. The system mainly consists of three core processes: health check, image upload, and prediction, which interact with both the user and the image storage component. The health check process is responsible for verifying whether the backend server and API services are functioning correctly by receiving a request from the user and returning a status response.

The image upload process allows users to upload images into the system. Once uploaded, the image is stored in the image storage component, and the system returns the upload status along with the public URI of the stored image. In the prediction process, the uploaded image is passed to the CON-KAN model for classification. The model processes the image and predicts whether it belongs to the cat or dog category. Finally, the prediction result is returned to the user through the backend response system in real time.

Schema



Fig. 5: Schema for endpoint

Project Snapshots

Backend Fast API

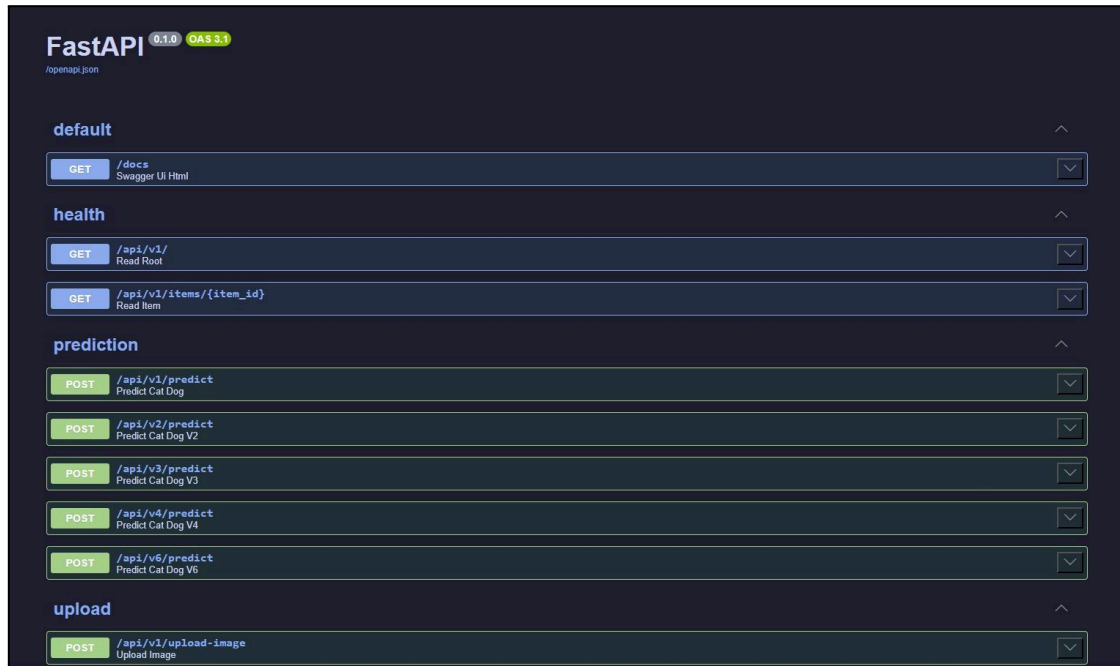


Fig. 6: Backend Fast API routes

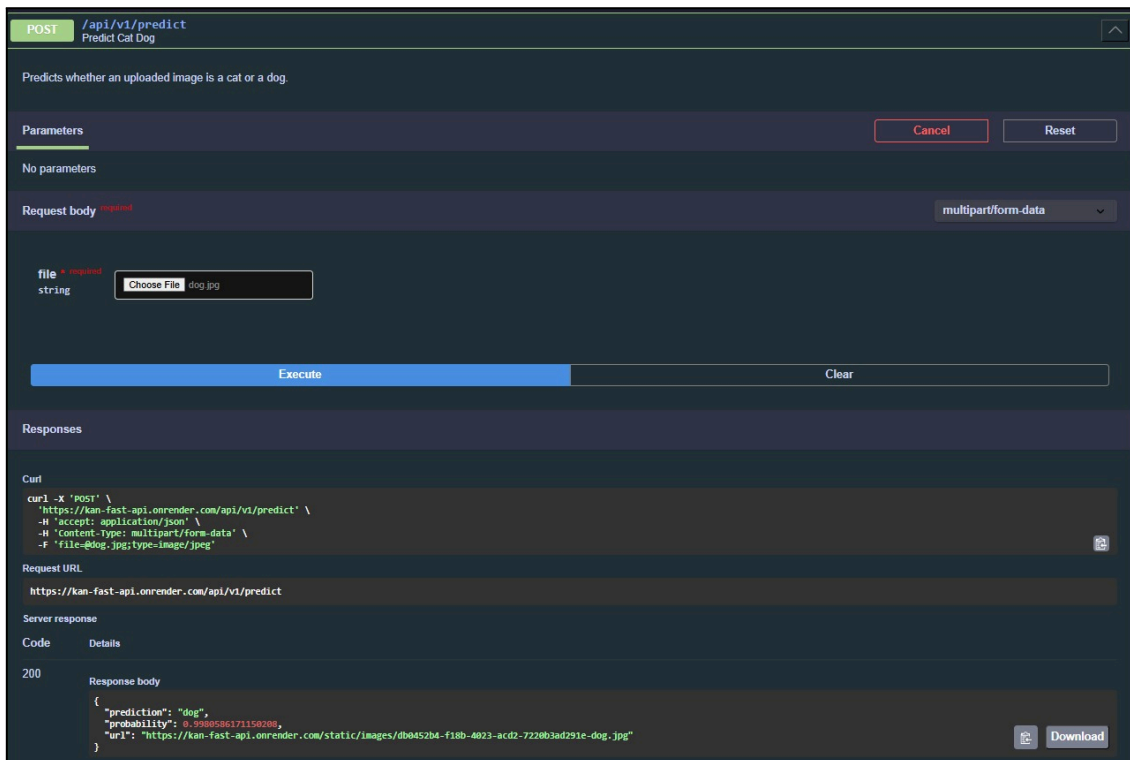


Fig. 7: CNN model Testing Route

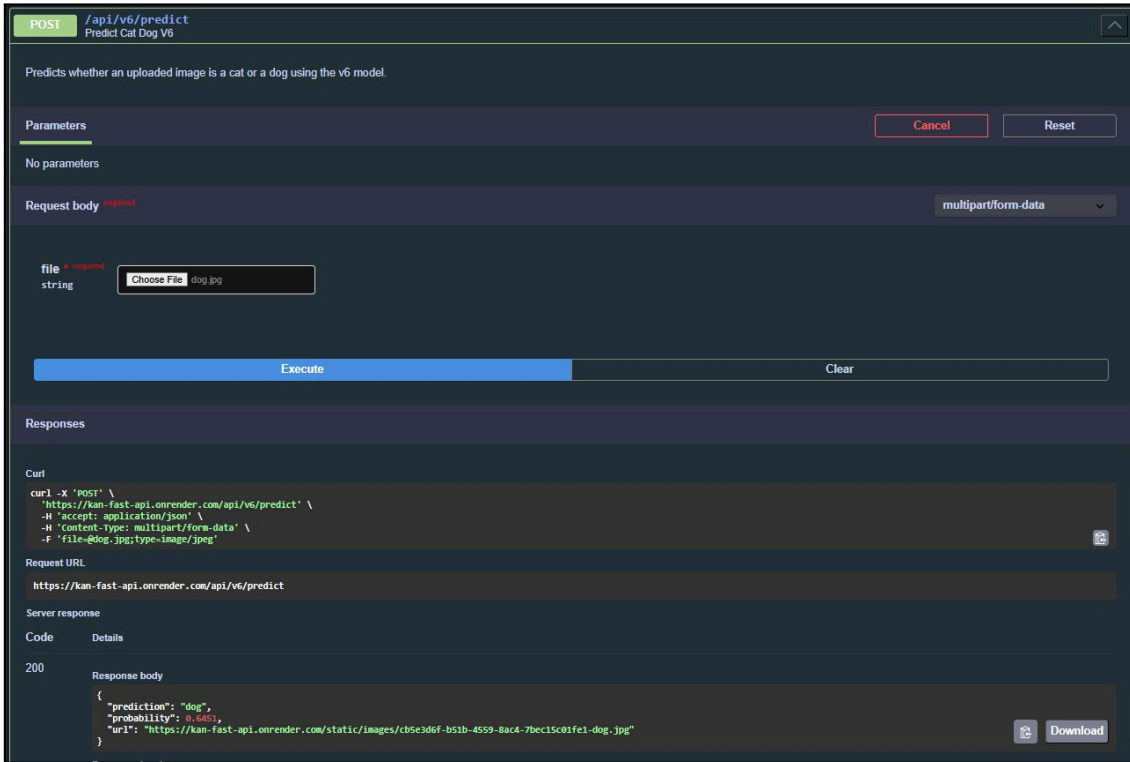


Fig. 8: CON KAN model Testing Route

Frontend

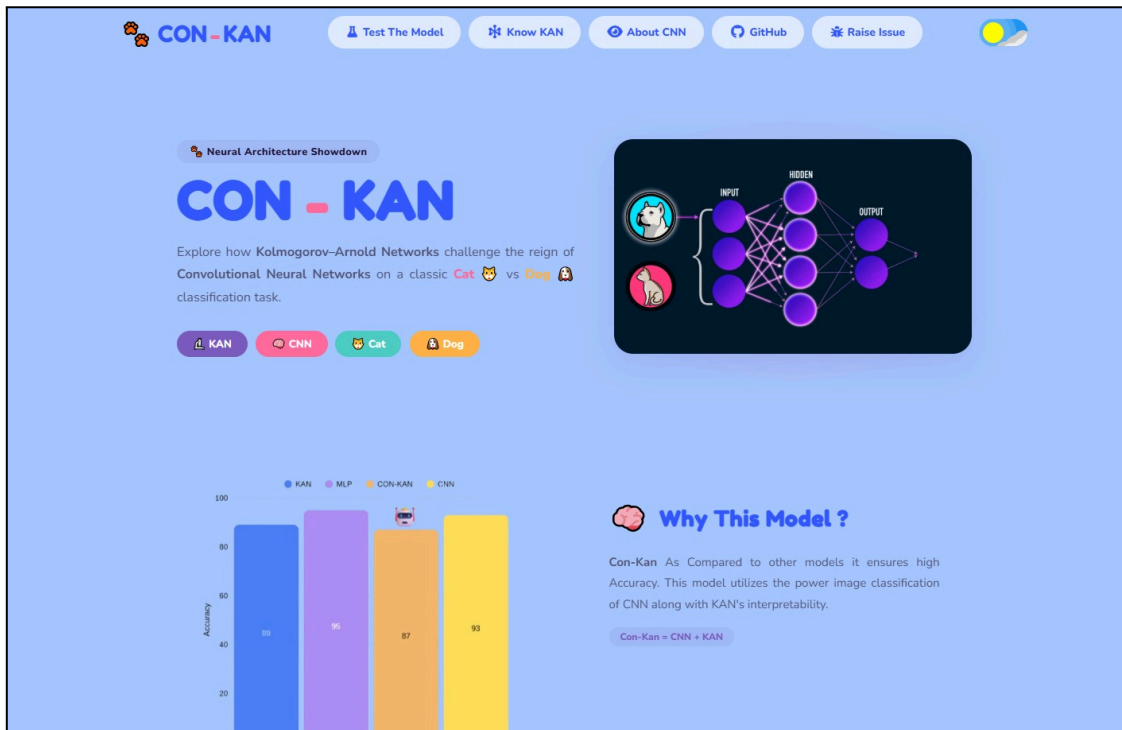


Fig. 9: Frontend Landing page



Fig. 10: Reduced loss while Training

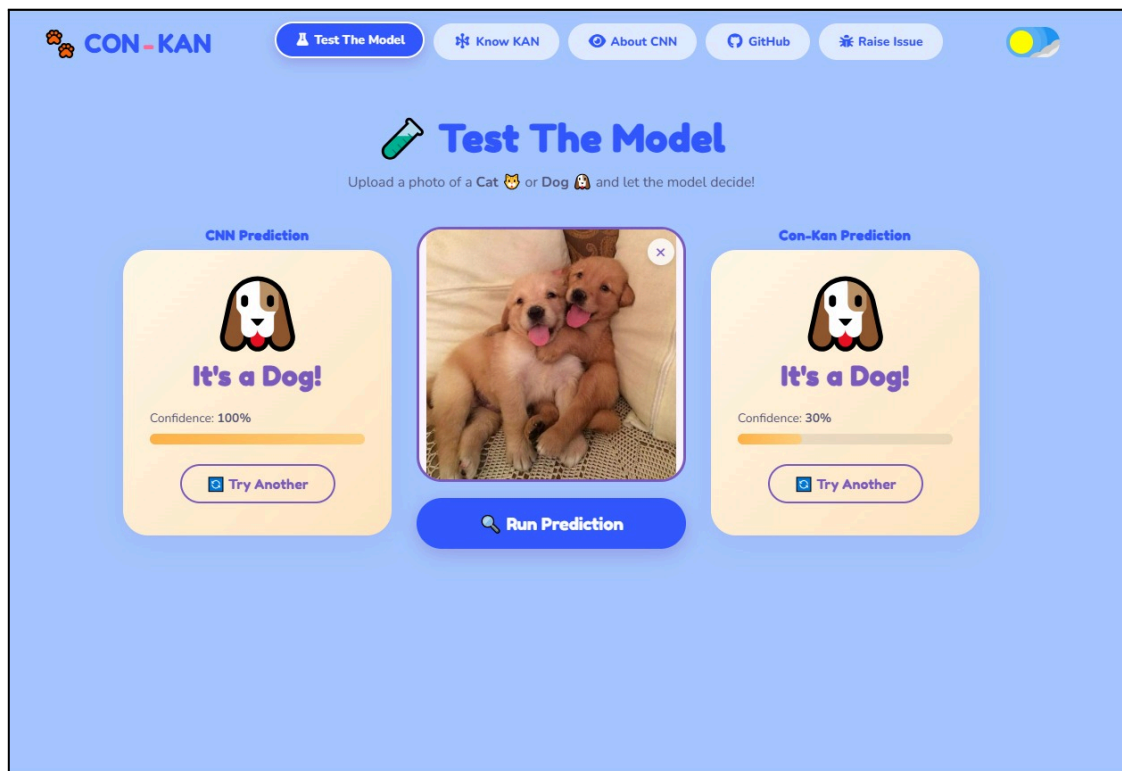


Fig. 11: Comparative analysis of the model prediction

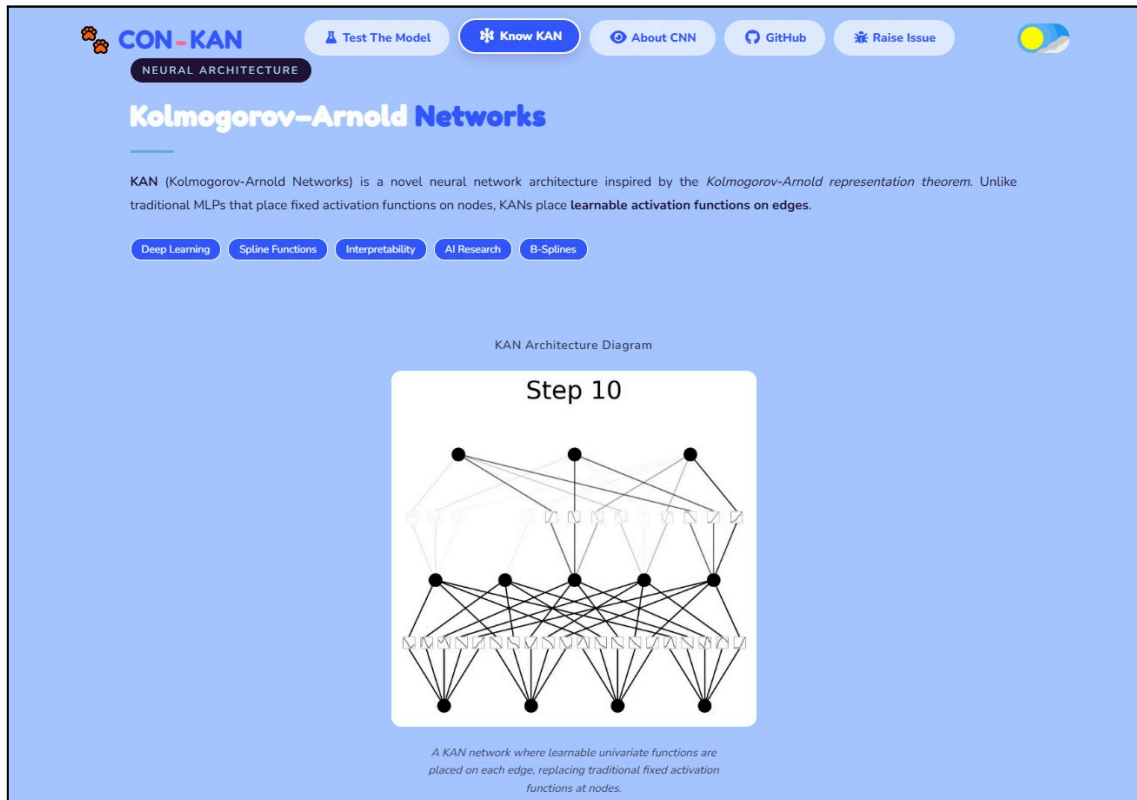


Fig. 12: Study of Kolmogorov Arnold Networks

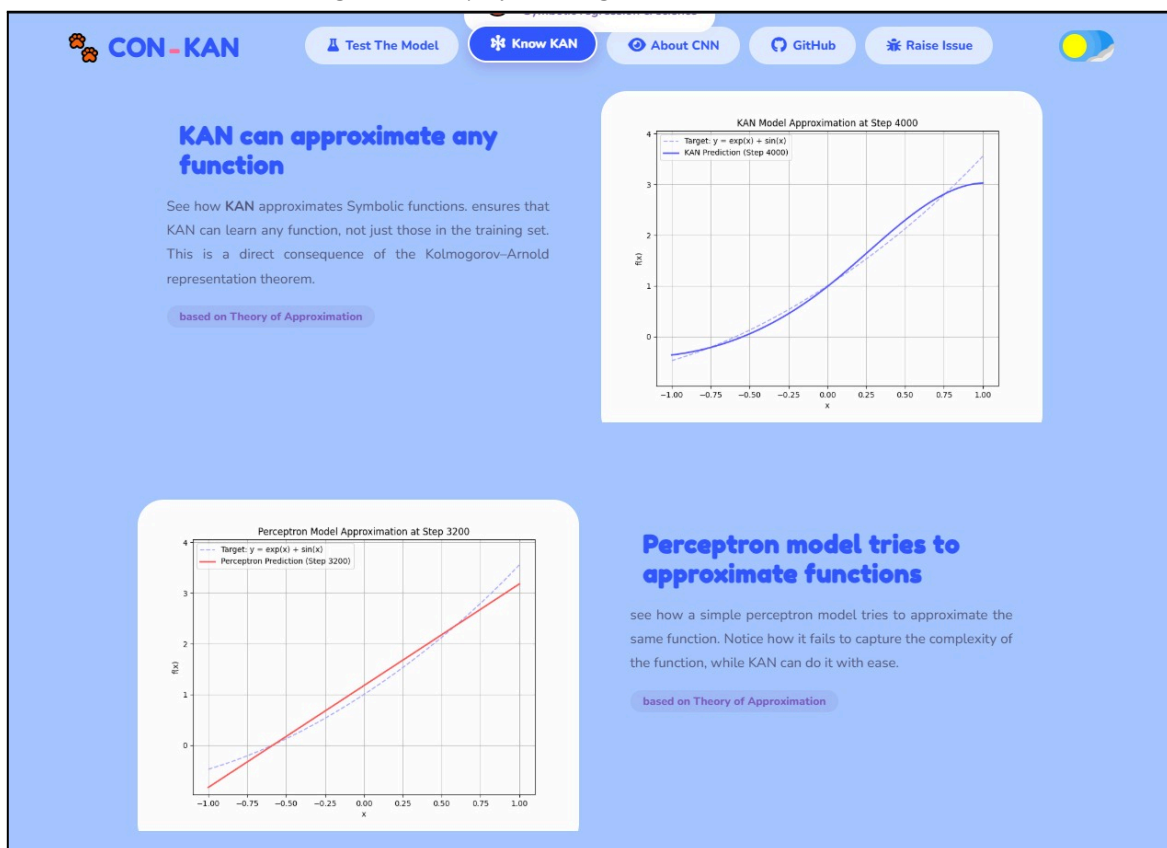


Fig. 13: KAN Approximation Results



Fig. 14: KAN Advantages

Model Results

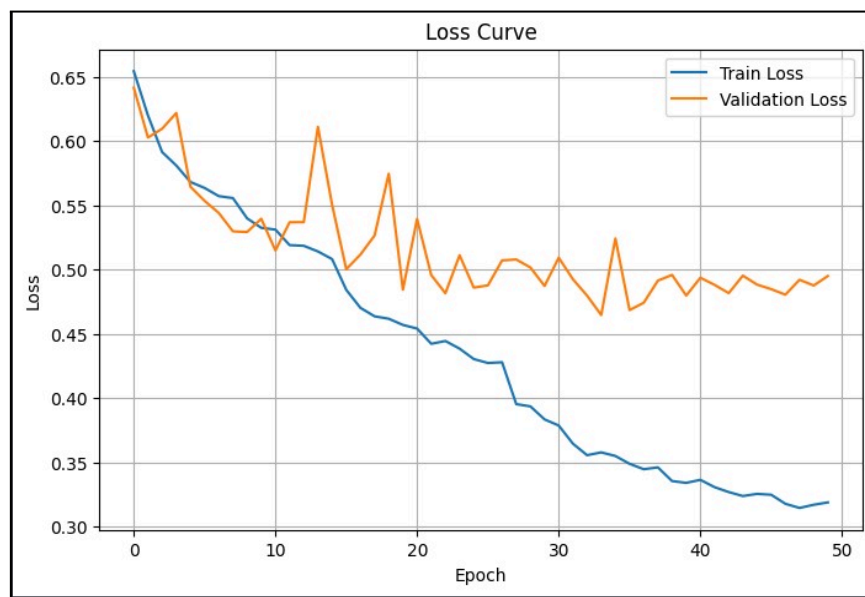


Fig. 15: Loss Curve Training and Validation results

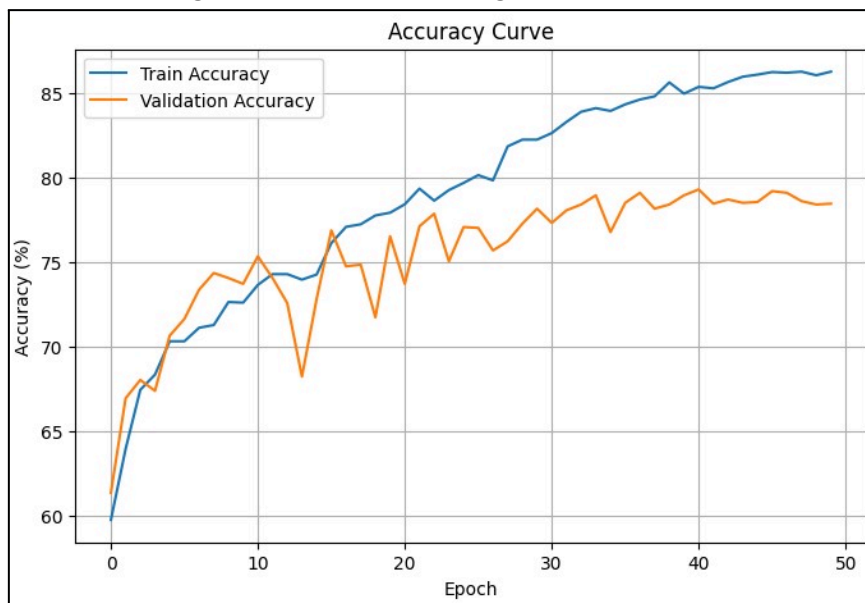


Fig. 16: Accuracy Curve Training and Validation results

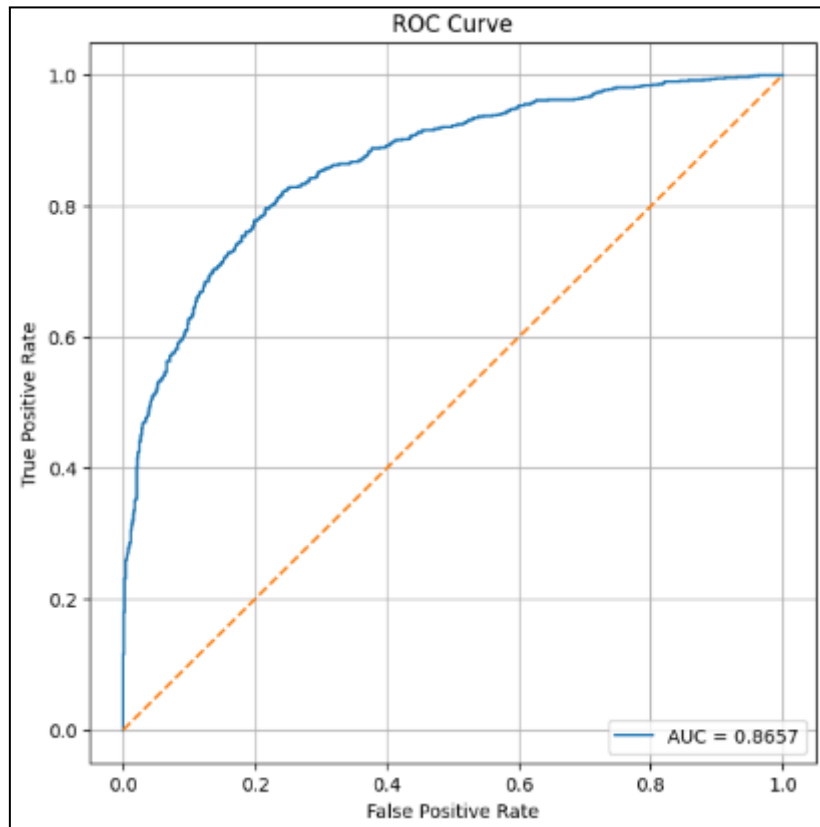


Fig. 17: ROC AUC Curve

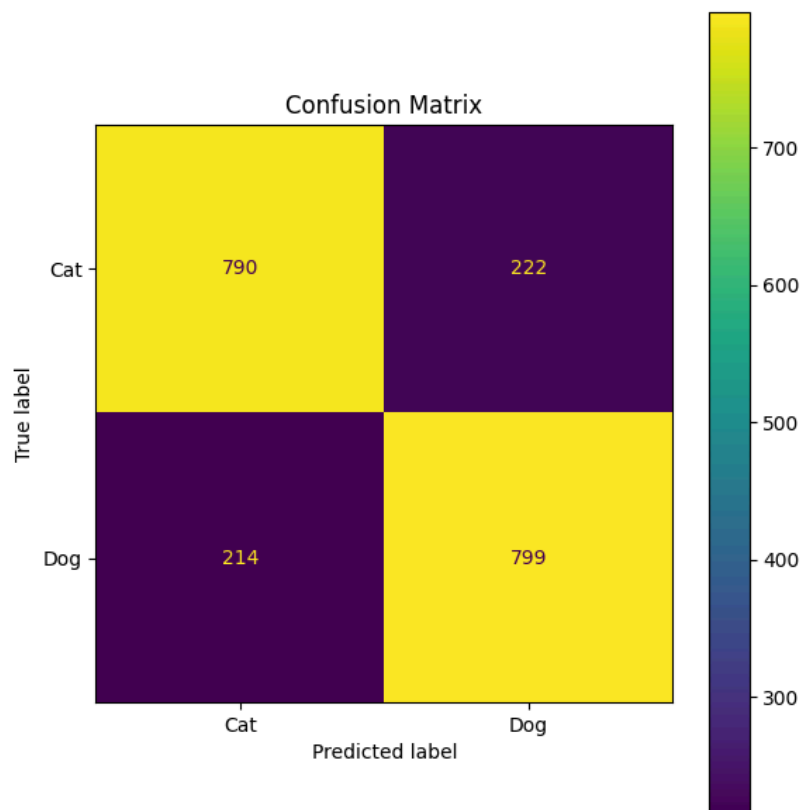


Fig. 18: Confusion Matrix

Selected Codes

Model Architecture

kan_model.py

```
import torch
import torch.nn as nn

def b_spline_basis(x, knots, k):
    B = ((x[:, None] >= knots[:-1]) &
          (x[:, None] < knots[1:])).float()
    for d in range(1, k + 1):
        left_num = x[:, None] - knots[:-d-1]
        left_den = knots[d:-1] - knots[:-d-1]

        right_num = knots[d+1:] - x[:, None]
        right_den = knots[d+1:] - knots[1:-d]

        left = torch.where(
            left_den != 0,
            left_num / left_den,
            torch.zeros_like(left_den)
        )
        right = torch.where(
            right_den != 0,
            right_num / right_den,
            torch.zeros_like(right_den)
        )
        B = left * B[:, :-1] + right * B[:, 1:]
    return B

class KANLayer(nn.Module):
    def __init__(self, in_dim, out_dim, num_knots, k=3):
        super().__init__()
        self.in_dim = in_dim
```

```

self.out_dim = out_dim
self.k = k
base_knots = torch.linspace(-1, 1, num_knots)
h = base_knots[1] - base_knots[0]
left_ext = base_knots[0] - h * torch.arange(k, 0, -1)
right_ext = base_knots[-1] + h * torch.arange(1, k + 1)
self.register_buffer(
    'knots',
    torch.cat([left_ext, base_knots, right_ext])
)
self.n_basis = len(self.knots) - k - 1
self.coef = nn.Parameter(
    torch.randn(
        in_dim,
        out_dim,
        self.n_basis
    ) * 0.001
)

def forward(self, x):
    B_all = []
    for i in range(self.in_dim):
        B_i = b_spline_basis(
            x[:, i],
            self.knots,
            self
        )
        B_all.append(B_i)
    B = torch.stack(B_all, dim=1)
    y = torch.einsum(
        'bik,ijk->bj',
        B,
        self.coef
    )
    return y

```



```

class MultiKAN(nn.Module):
    def __init__(self, width, num_knots=8, k=3):
        super().__init__()
        self.layers = nn.ModuleList()
        for i in range(len(width) - 1):
            self.layers.append(
                KANLayer(
                    width[i],
                    width[i + 1],
                    num_knots,
                    k
                )
            )
    def forward(self, x):
        for i, layer in enumerate(self.layers):
            x = layer(x)
            if i != len(self.layers) - 1:
                x = torch.nn.functional.leaky_relu(x, 0.1)
        return x

class MyNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1),
            nn.BatchNorm2d(16),
            nn.LeakyReLU(0.1),
            nn.Conv2d(16, 32, 3, padding=1),
            nn.BatchNorm2d(32),
            nn.LeakyReLU(0.1),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.1),
            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),

```

```

        nn.LeakyReLU(0.1),
        nn.MaxPool2d(2, 2),

        nn.AdaptiveAvgPool2d((1, 1))
    )

    # PRE-KAN NORMALIZATION
    self.pre_kan = nn.Sequential(
        nn.BatchNorm1d(128),
        nn.Linear(128, 128),
        nn.Tanh()
    )

    # SMALLER KAN
    self.kan = MultiKAN(
        width=[128, 257, 1],
        num_knots=8,
        k=3
    )

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        x = self.pre_kan(x)
        x = self.kan(x)
        return x

```

Future Scope

The current implementation of the CON-KAN architecture provides a strong foundation for exploring hybrid deep learning systems that combine traditional convolutional feature extraction with KAN-based functional classification. Although the proposed system successfully demonstrates binary image classification within a resource-constrained deployment environment, there are several opportunities for extending the architecture, improving computational efficiency, and expanding the overall research potential of the model.

One of the major future directions of this project is the deeper integration of KAN architectures within convolutional neural networks. In the current implementation, KAN is used only in the final classification stage due to computational and deployment limitations. Future research can explore the possibility of integrating KAN-inspired operations into intermediate feature extraction stages or convolutional blocks themselves. Such experimentation may help determine whether functional mapping approaches can improve feature representation beyond the final classification layer.

Another important area for future improvement involves optimizing KAN operations for resource-constrained environments. Since spline-based functional learning introduces additional computational overhead compared to traditional dense layers, future work can focus on techniques such as model compression, pruning, quantization, and lightweight KAN implementations to improve inference speed on CPU-based systems and edge devices. This would make hybrid CON-KAN architectures more practical for real-world deployment scenarios.

The current system performs binary image classification using the Cats vs Dogs dataset. In future versions, the architecture can be extended to support multi-class image classification tasks using larger and more complex datasets such as CIFAR-10, CIFAR-100, or domain-specific custom datasets. Evaluating the performance of CON-KAN on more challenging classification problems would provide better insight into the scalability and adaptability of the architecture.

Future research can also include comparative analysis between CON-KAN and other modern deep learning architectures such as Vision Transformers (ViTs), EfficientNet, ResNet, and hybrid transformer-based systems. Such comparisons may help evaluate whether KAN-based classifiers provide advantages in terms of parameter efficiency, interpretability, functional representation, or generalization capability.

The deployment infrastructure can also be significantly improved in future implementations. Currently, the model is deployed using a FastAPI backend hosted on a CPU-based cloud server. Future versions can incorporate GPU-enabled cloud deployment, containerized inference pipelines using Docker, scalable orchestration systems, and optimized inference engines such as ONNX Runtime or TensorRT to improve scalability and inference performance.

In addition, the frontend system can be expanded into a more interactive AI experimentation platform. Features such as real-time prediction visualization, confidence score analysis,

model comparison dashboards, training metric visualization, and user-uploaded dataset testing can improve usability and transform the project into a more comprehensive deep learning research environment.

Another long-term research direction involves studying the mathematical behavior of KAN-based learning in image classification tasks. Since KAN architectures are relatively new and still under active research, future work can investigate spline function optimization, convergence behavior, parameter sensitivity, and functional representation learning in hybrid CNN-KAN systems.

Finally, the project can evolve into a broader experimental framework for non-traditional neural architectures. Future studies may explore combinations of CNNs, KANs, attention mechanisms, and transformer-based approaches to develop more efficient and expressive image classification systems. Such research could contribute to the growing exploration of alternative neural computation paradigms beyond conventional MLP-based architectures.

Overall, the future scope of the CON-KAN project extends beyond basic image classification and opens opportunities for advanced deep learning research, efficient AI deployment, explainable neural systems, and experimentation with next-generation functional neural architectures.

Conclusion

The CON-KAN project successfully demonstrates the design, implementation, and deployment of a hybrid deep learning architecture that combines the feature extraction capability of Convolutional Neural Networks (CNNs) with the functional learning behavior of Kolmogorov-Arnold Networks (KANs) for image classification tasks. The project was developed not only as a practical image classification system but also as a research-oriented exploration into alternative neural architectures beyond traditional CNN-MLP pipelines.

The proposed architecture utilizes multiple convolutional layers to extract hierarchical spatial features from input images while replacing the conventional fully connected classification stage with a KAN-based functional mapping layer. This hybrid approach was designed to investigate whether KAN can function as an effective alternative to dense neural layers while still maintaining practical deployment feasibility within resource-constrained environments.

The complete system was successfully implemented using an end-to-end frontend-backend inference workflow. The Angular-based frontend, FastAPI backend, and PyTorch model integration enabled real-time image upload, preprocessing, model inference, and prediction visualization through an interactive web interface. The deployment pipeline also demonstrated that the proposed architecture could operate effectively within a CPU-based cloud deployment environment hosted on Render.com, validating the feasibility of lightweight AI inference systems using hybrid architectures.

Experimental evaluation showed that the model achieved satisfactory classification performance on the Cats vs Dogs dataset while maintaining stable inference behavior and successful end-to-end deployment. Although the current CON-KAN implementation does not outperform highly optimized traditional CNN architectures, the obtained results remain promising because the hybrid model achieved comparatively competitive performance while introducing a fundamentally different classification mechanism through KAN-based functional mappings. These observations support the idea that KAN can serve as a viable alternative classification framework worthy of further research and optimization.

One of the major achievements of this project is the balance maintained between research experimentation and practical engineering constraints. Instead of developing a fully KAN-based convolutional architecture, which would introduce significant computational overhead, the system strategically integrates KAN only within the final classification stage. This design decision helped maintain manageable computational complexity, support CPU-based deployment, and still preserve the experimental objective of evaluating KAN-based learning behavior in image classification tasks.

The project also highlights the importance of exploring non-traditional neural network architectures in modern deep learning research. Most contemporary image classification systems continue to rely heavily on standard fully connected layers and conventional learning structures. By integrating KAN into a practical classification workflow, the CON-KAN architecture contributes toward ongoing research efforts focused on alternative functional learning paradigms and mathematically inspired neural representations.

From a software engineering perspective, the project successfully integrates multiple technologies into a unified AI system. The frontend interface, backend APIs, preprocessing pipeline, model inference workflow, and deployment infrastructure collectively demonstrate a complete AI application lifecycle rather than an isolated machine learning experiment. This combination of deep learning research, system integration, and deployment engineering increases the practical relevance of the project.

Another important outcome of this work is the understanding gained regarding the trade-offs between architectural innovation and computational feasibility. While KAN introduces interesting theoretical advantages in functional representation learning, practical implementation challenges such as computational overhead, inference cost, and deployment efficiency remain important considerations. The project therefore provides both experimental insights and practical observations regarding the real-world applicability of hybrid CON-KAN systems.

Overall, the CON-KAN project successfully fulfills its research and implementation objectives by demonstrating that hybrid CON-KAN architectures can be developed, trained, deployed, and evaluated effectively for image classification tasks. The system establishes a strong foundation for future experimentation involving KAN-based neural architectures, efficient AI deployment strategies, and alternative functional learning approaches within modern deep learning systems.

References

- KAN: Kolmogorov–Arnold Networks, Research Paper
 - <https://arxiv.org/pdf/2404.19756>
- Convolutional-KAN
 - <https://share.google/MepTJcKIJ11TvDtch>
- B-Spline On-Line Geometric Modeling Notes, Paper
 - <https://www.cs.unc.edu/~dm/UNC/COMP258/LECTURES/B-spline.pdf>
- B-Spline visualization tool
 - <https://b-spline-site.onrender.com/bspline.html>
- PyTorch
 - <https://pytorch.org/>
- Matplotlib Pyplot
 - https://matplotlib.org/3.5.3/api/_as_gen/matplotlib.pyplot.html
- NumPy
 - <https://numpy.org/>
- FastAPI
 - <https://fastapi.tiangolo.com/>
- Pillow
 - <https://pypi.org/project/pillow/>
- Torchvision
 - <https://docs.pytorch.org/vision/stable/index.html>
- Cat Dog Image Dataset
 - <https://www.kaggle.com/datasets/tongpython/cat-and-dog>
- Angular 13
 - <https://v13.angular.io/docs>
- TypeScript
 - <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>
- Font Awesome
 - <http://fontawesome.com/>